CarLoop Web

Plateforme web de services automobiles augmentée par l'intelligence artificielle

Rapport technique — Mini-projet « IA for Software Engineering »

ESPRIT — Année universitaire 2026-2027 — Classe 5SAE…

Akrem Argoubi · Firas Tourki · Aziz Chamekh · Yassine Jmal · Amine Kaabi

Dépôt : github.com/akremargoubi/CarloopAiForSoftware

Octobre 2026

Sommaire

- 1. Contexte et objectifs
- 2. Architecture globale et choix techniques
- 3. Service IA commun : intégration de l'API LLM
- 4. Les cinq modules IA
- 5. Choix, adaptation et optimisation des modèles
- 6. Évaluation de la performance et de la fiabilité
- 7. L'IA dans le développement : génération de code et productivité
- 8. Tests, détection d'anomalies et refactoring assistés par IA
- 9. Limites éthiques et techniques
- 10. Collaboration et dépôt Git
- 11. Conclusion et perspectives
- Annexe A — Documentations individuelles
- Annexe B — Lancer l'application

1. Contexte et objectifs

En Tunisie, un automobiliste qui a besoin d'une vidange, de pneus ou d'une réparation doit appeler plusieurs garages, sans connaître les prix ni savoir quel service réserver face à un bruit ou à un voyant. Peu de conducteurs suivent un plan d'entretien adapté à leur véhicule.

CarLoop Web réunit sur une seule plateforme les services automobiles (lavage, vidange, pneus, climatisation, électricité, carrosserie, mécanique lourde, dépannage) et met en relation trois profils :

- **Client** : recherche de garages, réservation et suivi, gestion de ses véhicules, avis après prestation.
- **Professionnel** : profil du garage et services (prix, durée), agenda des réservations, envoi de devis réels, réponses aux avis.
- **Administrateur** : validation des comptes pro, statistiques de la plateforme, suivi des appels IA et du taux d'erreur par module.

L'objectif du mini-projet est d'intégrer l'IA à chaque étape du parcours, avec **un module IA par membre** de l'équipe, et d'utiliser l'IA générative dans le développement lui-même (génération de code, tests, détection d'anomalies).

Membre	GitHub	Module IA	Partie web associée
Akrem Argoubi	akremargoubi	Recherche intelligente + socle IA	Socle, prestataires, recherche
Firas Tourki	firastourki	Chatbot d'aide au diagnostic	Widget de chat, historique
Aziz Chamekh	azizmikha	Estimation de devis	Réservations et devis
Yassine Jmal	—	Analyse des avis	Avis et réponses du pro
Amine Kaabi	kaabiamine	Plan d'entretien personnalisé	Mes véhicules, historique

2. Architecture globale et choix techniques

L'application est un monorepo npm (`server/` et `client/`) écrit entièrement en TypeScript. Elle est organisée autour d'un **cœur commun** et de **modules indépendants** : chaque fonctionnalité IA vit dans son propre dossier côté serveur et côté client, et peut être activée ou désactivée par une seule ligne dans un registre.



Choix	Justification
React 18 + Vite + TypeScript	Écosystème riche, typage partagé avec le serveur, build rapide.
Tailwind CSS	Charte CarLoop cohérente et responsive sans feuille de style à maintenir.
Node.js + Express + TypeScript	Même langage que le front, API REST légère.
MongoDB + Mongoose	Schéma souple, adapté au stockage des sorties JSON de l'IA (plans, résumés, estimations).
Zod	Valide toutes les entrées utilisateur ET chaque réponse du LLM.

Choix	Justification
OpenRouter	Une seule API (compatible OpenAI) pour des centaines de modèles : le modèle se change dans `.env` sans toucher au code.
Vitest + Supertest + MongoDB en mémoire	Tests unitaires et d'intégration exécutables sur n'importe quelle machine, sans base ni réseau.
Docker Compose	Lancement de MongoDB, de l'API et du front en une commande.

Originalité. (1) Architecture « cœur + modules » : les modules ne s'importent jamais entre eux et communiquent par l'API, les URL et des « slots » d'interface (le plan d'entretien ouvre la réservation pré-remplie ; le diagnostic ouvre la recherche de garages adaptés). (2) Un mode démo : sans clé API, chaque module renvoie une réponse simulée qui passe par la même validation que les vraies réponses. (3) Une IA présente dans tout le parcours client, de la description d'un symptôme jusqu'au plan d'entretien.

3. Service IA commun : intégration de l'API LLM

Aucun module n'appelle le LLM directement. Tous passent par la fonction `runAiTask(nom, promptSystème, message, schémaZod, options)` (dossier `server/src/ai/`) :

- **Anonymisation** : emails, téléphones tunisiens, plaques (« 215 TU 4821 »), adresses et noms connus sont remplacés par [EMAIL], [TELEPHONE], [PLAQUE], [ADRESSE], [NOM] avant l'envoi.
- **Garde-fou anti-injection** ajouté automatiquement à chaque prompt système.
- **Appel LLM** : JSON imposé (`response_format`), délai maximal configurable, un nouvel essai avec attente progressive sur erreur 429 ou 5xx, liste de modèles de secours.
- **Extraction et validation** : le JSON est extrait de la réponse puis validé par le schéma Zod du module, avec des règles de cohérence métier.
- **Relance corrective** : si la réponse est invalide, le LLM reçoit la liste exacte des erreurs et doit corriger (une fois).
- **Journal AiLog** : module, succès ou code d'erreur, durée, modèle, tokens, nombre de données masquées, sans aucun contenu personnel.

Erreur	HTTP	Cause	Affichage côté client
AI_TIMEOUT	504	Pas de réponse dans le délai	« L'assistant met trop de temps » + Réessayer
AI_QUOTA_EXCEEDED	429	Quota du fournisseur atteint	« Quota dépassé, réessayez plus tard »
AI_NOT_CONFIGURED	503	Clé absente et mode démo désactivé	« Service IA non configuré »
AI_INVALID_OUTPUT	502	Réponse non conforme après relance	« Réponse inexploitable » + Réessayer
AI_UPSTREAM_ERROR	502	Erreur du fournisseur	« Service momentanément indisponible »
RATE_LIMITED	429	Trop d'appels IA par minute	« Patientez une minute »

La clé OpenRouter reste exclusivement côté serveur (fichier `.env` exclu de Git) ; le navigateur ne communique jamais avec le fournisseur.

4. Les cinq modules IA

Chaque module fournit un prompt système versionné (`prompt.ts`), un schéma de sortie (`schema.ts`), une réponse simulée (`mock.ts`) et ses tests. Les prompts complets, avec exemples d'entrée et de sortie réelles, sont

dans `docs/prompts.md`.

Module	Endpoint	Entrée → sortie	Contrôles spécifiques
1. Diagnostic (Firas)	POST /api/ai/diagnostic	Symptôme + véhicule + 10 derniers messages → causes probables, urgence, service recommandé, conseil de sécurité	Catégorie dans la liste CarLoop ; conversations sauvegardées
2. Recherche (Akrem)	POST /api/ai/search	Phrase libre → filtres (catégorie, ville, budget, disponibilité) → classement justifié	Seuls les garages réellement trouvés sont gardés ; filtres relâchés si aucun résultat
3. Devis (Aziz)	POST /api/ai/quote-estimate	Véhicule + service → fourchette de prix, pièces, main d'œuvre, hypothèses	Prix positifs, prix_min ≤ prix_max ; prix catalogue du garage en référence
4. Avis (Yassine)	POST /api/ai/reviews-summary, /review-reply	Avis d'un garage → résumé, sentiment, points forts / faibles ; 2 réponses pour le pro	Résumé mis en cache ; nom du client masqué
5. Entretien (Amine)	POST /api/ai/maintenance-plan/:id	Véhicule + usage + historique → échéances en km et date, priorités, conseils	Échéance ≥ kilométrage actuel et ≥ date du jour

5. Choix, adaptation et optimisation des modèles

5.1 Pourquoi un LLM généraliste via OpenRouter

Les cinq tâches sont des tâches de langage (comprendre une demande libre, raisonner sur un symptôme, résumer, rédiger en français). Un LLM généraliste piloté par prompt engineering les couvre toutes avec une seule intégration à sécuriser. OpenRouter permet de comparer et de changer de modèle par simple configuration.

5.2 Comparaison de modèles

Le script `npm run eval -w server` joue 12 cas annotés (5 diagnostics dont une tentative d'injection, 3 extractions de recherche, 1 devis, 2 cas d'avis, 1 plan d'entretien) avec les mêmes prompts et schémas que l'application.

Modèle	JSON valide	Correct	Latence médiane	Tokens moyens (entrée / sortie)
Règles (mode démo)	100 %	83 %	< 1 ms	—
nvidia/nemotron-3-super-120b-a12b:free	100 %	100 %	11,6 s	730 / 841
google/gemma-4-26b-a4b-it:free	8 %	8 %	—	indisponible (429 du fournisseur)
google/gemma-4-31b-it:free	0 %	0 %	—	indisponible (429 du fournisseur)

Décision : nemotron-3-super-120b en modèle principal, gemma-4-31b en secours (`OPENROUTER_MODEL=nvidia/nemotron-3-super-120b-a12b:free,google/gemma-4-31b-it:free`). Le mode démo à base de règles se trompe sur 2 diagnostics sur 5 (« ne démarre plus », « panne sur autoroute ») : il ne comprend pas le contexte, ce qui justifie l'usage du LLM.

5.3 Techniques d'adaptation et d'optimisation appliquées

Technique	Mise en œuvre dans CarLoop
Prompt tuning	Prompts structurés (rôle, tâche, règles, format JSON exact, listes fermées) ; règles ajoutées pour éviter d'inventer ville ou budget, pour traiter « pas cher » comme un critère, pour garder les échéances dans le futur.
Réglage des hyperparamètres	Température 0 pour l'extraction (déterministe), 0,2–0,3 pour l'analyse, 0,6 pour rédiger des réponses variées ; `max_tokens` ajusté par module ; effort de raisonnement réduit (`reasoning.effort = low`).
Choix et comparaison de modèles	Évaluation de 3 modèles + règles (5.2) ; modèle principal + secours.
Contexte injecté (proche du RAG)	Les données utiles sont récupérées en base puis données au LLM : garages candidats (recherche), prix catalogue (devis), historique d'entretien (plan), avis (résumé). Le LLM ne travaille que sur des données réelles.
Cache	Résumé des avis mis en cache et régénéré uniquement si un nouvel avis arrive.
Réduction du coût et des tokens	Historique du chatbot limité à 10 messages, 40 avis et 30 entretiens au maximum, recherche en deux étapes (MongoDB filtre, le LLM ne classe que 8 candidats), modèles gratuits.
Latence	Délais alignés sur la latence mesurée (serveur 90 s, front 180 s), indicateurs de chargement, modèle rapide payant possible sans changer le code.
Fiabilité	Validation Zod + relance corrective, nouvel essai sur 429/5xx, modèles de secours, mode démo de repli.

Le fine-tuning n'a pas été retenu : les modèles généralistes atteignent 100 % sur notre jeu de test grâce au prompt engineering et à la validation, et un fine-tuning demanderait un jeu de données annoté important et un coût d'hébergement incompatibles avec le projet.

6. Évaluation de la performance et de la fiabilité

En plus du jeu de 12 cas (section 5.2, détail dans `docs/EVALUATION.md`), l'application complète a été testée le 18/10/2026 avec la vraie clé OpenRouter, sur la base de démonstration :

Module	Requête	Résultat	Durée
Diagnostic	« voyant moteur orange clignote, la voiture tremble » (Peugeot 208)	Ratés d'allumage (bougies, bobine), urgence critique, service mécanique, conseil de remorquage	10 s
Recherche	« vidange pas chère à Ariana samedi matin »	Filtres exacts + critère « pas cher », garage classé avec justification factuelle	18 s
Devis	Plaquettes avant, Clio diesel 2015	130–220 TND, 1,2 h, hypothèses explicites	27 s
Plan d'entretien	Peugeot 208 urbaine, 68 000 km	6 interventions datées, retards détectés (vidange, batterie)	131 s

Problèmes détectés par l'évaluation et corrigés : (1) JSON coupé sur le plan d'entretien, le modèle à raisonnement consommant tous les tokens autorisés → effort de raisonnement réduit et limite relevée, puis 12/12 ; (2) délais trop courts par rapport à la latence réelle → délais réalignés ; (3) indisponibilité des modèles gratuits → modèle de secours. Une erreur 502 passagère du fournisseur a aussi été observée puis absorbée par un nouvel essai.

7. L'IA dans le développement : génération de code et productivité

L'assistant de code **Claude Code** (Anthropic) a été utilisé en continu, dans VS Code, comme un développeur pair : lecture du dépôt, génération de code dans le style existant, exécution des tests, correction des erreurs, rédaction de la documentation. Chaque proposition a été relue, testée et validée avant d'être commitée.

Tâche	Usage de l'IA	Contrôle humain
Structure du projet	Proposition de l'architecture cœur + modules et des contrats `FeatureModule` / `ClientFeature`	Validation des choix, adaptation au découpage de l'équipe
Code des modules	Génération des routes, services, schémas Zod, prompts, mocks et pages React à partir de la description fonctionnelle	Relecture, tests, ajustements du métier tunisien (prix TND, villes, plaques)
Prompts	Rédaction et itération des prompts système, puis évaluation automatique	Choix des règles et des cas de test
Tests	Écriture des 131 tests, y compris les cas d'erreur du LLM	Vérification que chaque test échoue si la règle est cassée
Documentation	README, référence API, documentation IA, prompts, évaluation	Relecture et correction

Gain de productivité observé. Le dépôt compte environ 8 500 lignes de code applicatif et 1 600 lignes de tests. À titre d'exemple, le module 5 côté client (2 pages, formulaire, carte du plan, test) et ses 13 tests serveur ont été produits et vérifiés en une seule session de travail ; la revue de sécurité, le script d'évaluation, leurs corrections et la documentation associée en une après-midi. Le gain le plus net concerne le code répétitif (CRUD, validation, tests d'API) et la documentation ; le temps humain se concentre sur les choix d'architecture, la vérification et les décisions métier.

Limites constatées : l'IA peut proposer du code plausible mais inexact (corrigé grâce aux tests), elle ne connaît pas les contraintes de l'équipe sans contexte (d'où le fichier `CLAUDE.md` de règles communes), et ses affirmations doivent être vérifiées dans le code.

8. Tests, détection d'anomalies et refactoring assistés par IA

8.1 Tests générés avec l'IA

131 tests automatisés, tous au vert (109 côté serveur, 22 côté client), exécutables par `npm test` sans base de données ni réseau (MongoDB en mémoire, `fetch` simulé).

- **Unitaires (couche IA)** : anonymisation (19 cas), extraction JSON, nouvel essai sur 429/5xx, timeout, relance corrective, garde-fou anti-injection, modèles de secours.
- **Unitaires (schémas)** : pour chaque module, une sortie valide est acceptée, une sortie incohérente est rejetée, et la réponse simulée respecte le schéma.
- **Intégration (API)** : parcours métier complets, contrôle d'accès par rôle et par propriétaire, réponse invalide du LLM → 502 après relance, vérification que le message envoyé au LLM ne contient aucune donnée personnelle (plaque, nom, téléphone, coût).
- **Front** : affichage des erreurs IA, cartes de résultats, liens de réservation, filtrage des redirections.

8.2 Anomalies et vulnérabilités détectées

Revue de code orientée sécurité réalisée avec Claude Code, complétée par `npm audit`, l'évaluation automatique et les tests en conditions réelles (détail : `docs/QUALITE-SECURITE.md`).

#	Constat	Gravité	Correction	Vérification
F1	Redirection ouverte après connexion (`?redirect=//site-pirate.com`), aggravée par une CVE de React Router	Moyenne	Seuls les chemins internes sont acceptés	Test unitaire
F2	Aucune limite de tentatives sur la connexion (force brute)	Moyenne	10 échecs par IP / 15 min	Test d'intégration
F3	Injection de prompt possible via le texte utilisateur	Moyenne	Garde-fou ajouté à tous les prompts	Test + cas d'évaluation réussi
F4	React Router 6 vulnérable (npm audit)	Moyenne	Neutralisée par F1 ; migration v7 prévue	—
F5	Vulnérabilités d'outils de développement	Faible	Absents de l'image de production	npm audit --omit=dev
F6	JSON tronqué par le raisonnement caché du modèle	Fiabilité	Effort de raisonnement réduit, limite relevée	Évaluation 12/12
F7	Délais serveur / front / nginx inférieurs à la latence réelle	Fiabilité	Délais réalignés	Test réel
F8	Erreurs du fournisseur non journalisées	Maintenabilité	Journal serveur sans donnée sensible	Test réel
F9	Déploiement Docker incomplet	Déploiement	docker-compose.yml ajouté	docker compose config
F10	Module 5 sans tests	Couverture	13 tests ajoutés	npm test

Points vérifiés sans anomalie : injection NoSQL (toutes les entrées passent par Zod), contrôle d'accès entre clients, hash des mots de passe jamais renvoyé, clé API jamais exposée, aucune trace d'erreur interne renvoyée au client.

8.3 Recommandations de refactoring

- **Fait** : centralisation de tous les appels LLM dans `runAiTask` ; garde-fou anti-injection placé dans cette fonction plutôt que dans chaque prompt ; modèle configurable avec secours.
- **À faire** : migration vers React Router 7 ; formatage Prettier de tout le dépôt ; réponses simulées du plan d'entretien moins alarmistes.

9. Limites éthiques et techniques

- **Données personnelles** : minimisation (chaque module n'envoie que le nécessaire) et anonymisation automatique ; journal sans contenu. Limite : l'anonymisation par motifs peut manquer un format inhabituel.
- **Hallucinations** : sorties validées par Zod, listes fermées, identifiants vérifiés (un garage inventé est rejeté), avertissements visibles (« seul le devis du professionnel fait foi », « ne remplace pas un mécanicien »).
- **Sécurité des usagers** : le diagnostic signale l'urgence et donne un conseil de sécurité, sans jamais proposer de réparation dangereuse ni de diagnostic définitif.
- **Biais** : les estimations de prix reflètent ce que le modèle a appris et peuvent être éloignées du marché local ; elles sont ancrées sur le prix catalogue du garage quand il existe.
- **Dépendance au fournisseur** : quotas (50 requêtes/jour sur le compte gratuit), indisponibilités et latence (jusqu'à 2 minutes) ; atténuées par les modèles de secours, le mode démo et la possibilité de passer à un modèle payant rapide.
- **Transparence** : les contenus générés par l'IA sont signalés comme tels dans l'interface (« Estimation IA », « Plan d'entretien IA »).

10. Collaboration et dépôt Git

Dépôt : github.com/akremargoubi/CarloopAiForSoftware. Conventions : Conventional Commits (`feat`, `fix`, `test`, `docs`), une branche par module puis Pull Request, build et tests vérifiés après chaque fusion. Les règles communes (architecture, sécurité, tests) sont écrites dans `CLAUDE.md` pour que chaque membre, et les assistants IA qu'il utilise, produisent un code homogène.

[À compléter par l'équipe : branches et Pull Requests de chaque membre, captures de l'historique Git, répartition effective des tâches.]

11. Conclusion et perspectives

CarLoop Web intègre cinq modules IA dans un parcours cohérent, sur un socle commun qui rend chaque appel au LLM anonymisé, validé, journalisé et résistant aux pannes. L'évaluation montre 100 % de réponses valides et correctes sur notre jeu de test avec un modèle gratuit, et l'IA a été utilisée tout au long du développement pour écrire le code, les tests et détecter des failles réelles, corrigées et vérifiées.

Perspectives : élargir le jeu d'évaluation (une vingtaine de cas par module), comparer un modèle payant rapide, migrer vers React Router 7, ajouter le suivi en temps réel des interventions et une version mobile.

Annexe A — Documentations individuelles

Chaque membre présente sa partie : prompts et interactions avec les LLM utilisés pour développer, justification du modèle et des choix, gain de productivité.

Firas Tourki — Module 1 : chatbot d'aide au diagnostic

Module. Widget de chat flottant sur toutes les pages ; conversation multi-tours (10 derniers messages) ; sortie validée : causes probables avec probabilité, urgence (faible / moyenne / critique), service recommandé parmi les 8 catégories, conseil de sécurité ; bouton « Trouver un prestataire » vers la recherche ; conversations sauvegardées pour les utilisateurs connectés.

Résultats. 5/5 cas de diagnostic corrects avec nemotron-3-super-120b (dont la tentative d'injection), contre 3/5 pour les règles du mode démo ; 10 s en conditions réelles.

Exemples d'interactions avec Claude Code (session du 18/10/2026) :

- « ouvre le projet carloop et continue le travail » → analyse du dépôt, détection d'un module interrompu, écriture des tests et de l'interface manquants.
- « t'as tout fait selon la demande de prof ? » (avec la grille d'évaluation) → comparaison critère par critère, liste des manques (évaluation, documentation, Git).
- « fait tout ce qu'il faut selon ce qu'elle a demandé » → revue de sécurité (3 failles corrigées), script d'évaluation des modèles, tests en conditions réelles, documentation.

Prompts et interactions avec les LLM (outils de développement)

[À compléter par Firas Tourki : 3 à 5 prompts réellement envoyés à Claude / Copilot / ChatGPT pendant le développement du module (texte du prompt, ce que l'outil a produit, ce qui a été gardé ou corrigé).]

Justification du modèle et des choix

[À compléter par Firas Tourki : pourquoi ce prompt, cette température, ces contrôles Zod ; résultats du module dans docs/EVALUATION.md.]

Gain de productivité

[À compléter par Firas Tourki : temps estimé avec et sans IA, tâches accélérées, limites rencontrées.]

Akrem Argoubi — Module 2 : recherche intelligente et socle IA

Prompts et interactions avec les LLM (outils de développement)

[À compléter par Akrem Argoubi : 3 à 5 prompts réellement envoyés à Claude / Copilot / ChatGPT pendant le développement du module (texte du prompt, ce que l'outil a produit, ce qui a été gardé ou corrigé).]

Justification du modèle et des choix

[À compléter par Akrem Argoubi : pourquoi ce prompt, cette température, ces contrôles Zod ; résultats du module dans docs/EVALUATION.md.]

Gain de productivité

[À compléter par Akrem Argoubi : temps estimé avec et sans IA, tâches accélérées, limites rencontrées.]

Aziz Chamekh — Module 3 : estimation de devis

Prompts et interactions avec les LLM (outils de développement)

[À compléter par Aziz Chamekh : 3 à 5 prompts réellement envoyés à Claude / Copilot / ChatGPT pendant le développement du module (texte du prompt, ce que l'outil a produit, ce qui a été gardé ou corrigé).]

Justification du modèle et des choix

[À compléter par Aziz Chamekh : pourquoi ce prompt, cette température, ces contrôles Zod ; résultats du module dans docs/EVALUATION.md.]

Gain de productivité

[À compléter par Aziz Chamekh : temps estimé avec et sans IA, tâches accélérées, limites rencontrées.]

Yassine Jmal — Module 4 : analyse des avis

Prompts et interactions avec les LLM (outils de développement)

[À compléter par Yassine Jmal : 3 à 5 prompts réellement envoyés à Claude / Copilot / ChatGPT pendant le développement du module (texte du prompt, ce que l'outil a produit, ce qui a été gardé ou corrigé).]

Justification du modèle et des choix

[À compléter par Yassine Jmal : pourquoi ce prompt, cette température, ces contrôles Zod ; résultats du module dans docs/EVALUATION.md.]

Gain de productivité

[À compléter par Yassine Jmal : temps estimé avec et sans IA, tâches accélérées, limites rencontrées.]

Amine Kaabi — Module 5 : plan d'entretien personnalisé

Prompts et interactions avec les LLM (outils de développement)

[À compléter par Amine Kaabi : 3 à 5 prompts réellement envoyés à Claude / Copilot / ChatGPT pendant le développement du module (texte du prompt, ce que l'outil a produit, ce qui a été gardé ou corrigé).]

Justification du modèle et des choix

[À compléter par Amine Kaabi : pourquoi ce prompt, cette température, ces contrôles Zod ; résultats du module dans docs/EVALUATION.md.]

Gain de productivité

[À compléter par Amine Kaabi : temps estimé avec et sans IA, tâches accélérées, limites rencontrées.]

Annexe B — Lancer l'application

```
npm install
cp server/.env.example server/.env    # renseigner JWT_SECRET et OPENROUTER_API_KEY
npm run seed                          # données de démonstration (MongoDB requis)
npm run dev                           # API http://localhost:4000 · front http://localhost:5173

# ou, tout-en-un :
JWT_SECRET=... OPENROUTER_API_KEY=... docker compose up --build

npm test                              # 131 tests
npm run eval -w server -- <modèles>   # évaluation des modules IA
```

Rôle	Email	Mot de passe
Admin	admin@carloop.tn	Admin123!
Client	client1@carloop.tn … client5@carloop.tn	Client123!
Pro	pro1@carloop.tn … pro6@carloop.tn	Pro12345!